Crypto 1 SIV Pipeline

Gotta Go Low

This challenge is an example of performing a Low Exponent Attack on RSA.

Why It Works

When the exponent value, `e`, is small and the value of the `ciphertext` is smaller than the value of the modulus `n`, the plaintext can be recovered using the nth root of the ciphertext, where the `nth root` is the value of `e`.

The formula for RSA is `ciphertext = plaintext ^ e modulo n`. When `plaintext ^ e` < `n`, the ciphertext becomes equal to `plaintext ^ e`. Therefore, the `e root` of the ciphertext is the plaintext.

Finding The Flag

The flag can be found with only the values given in `output.txt`. Knowing the value of `n` is only necessary for knowing it is greater than the value of the `ciphertext`. Using a Python module such as `gmpy2` to calculate the e-th root of the ciphertext gives us the flag, `SVBGR{l0w_3xp0n3nt5_@r3_n0t_s@fe}`.

Challenge Code

```
import random
from math import gcd
import sympy

def genKeypair(p, q):
    n = p * q
    phi = (p - 1) * (q - 1)
    print(n)
    e = 3

    def modinv(a, m):
        m0, x0, x1 = m, 0, 1
        while a > 1:
            q = a // m
```

```
            m, a = a % m, m
            x0, x1 = x1 - q * x0, x0
        return x1 + m0 if x1 < 0 else x1

    d = modinv(e, phi)
    return ((e, n), (d, n))

def encrypt(pubKey, plaintext):
    e, n = pubKey
    plaintextInt = int.from_bytes(plaintext.encode(), byteorder='big')
    encrypted = pow(plaintextInt, e, n)
    return encrypted

def genPrime(bits):
    while True:
        possible = random.getrandbits(bits)
        possible |= (1 << bits - 1) | 1

        if sympy.isprime(possible):
            return possible

if __name__ == "__main__":
    while True:
        p = genPrime(512)
        q = genPrime(512)

        if gcd(3, (p - 1) * (q - 1)) == 1:
            break

    pubKey, privKey = genKeypair(p, q)
    message = "SVBGR{test_flag}"
    encrypted = encrypt(pubKey, message)
    print("Encrypted message:", encrypted)
```

Solve Code

```
import gmpy2

e = 3
n = <provided n>
```

```
ciphertext = <provided ciphertext>

plaintext_root, exact = gmpy2.iroot(ciphertext, e)

if exact:
    print("Exact cube root found.")
    plaintext_int = int(plaintext_root)
    plaintext_bytes = plaintext_int.to_bytes((plaintext_int.bit_length() + 7)
// 8, byteorder='big')
    print("Flag:", plaintext_bytes.decode())

else:
    print("Root is not exact - attack conditions not met.")
```

Provided Output.txt

```
e = 3
n =
131568056653373132012174976653266884910157447726840322128654668864744046838266
089026781586223439349724120314053694539817871939811571791816723493939318461523
177171366268168393668921342560692769288416456729904590430725433093936110904690
901655852707387030375716854722258158043345187159940346383427399753323791427
ciphertext =
898564915277349210856325643177982880844269990070750993964886895279898673815668
084088711509416748167698104435154155125903563814943672577759197896689419072923
530272379905743352154731864706846939063378835946564725599528080721144587149407
333
```